```
                (
                    [make] => Ibanez
                    [model] => RG-8570z
                )

            [1] => Array
                (
                    [make] => Fender
                    [model] => American Ultra Stratocaster
                )

        )

)
Array
(
    [0] => Array
        (
            [make] => Ibanez
            [model] => RG-8570z
        )

)
```

`$guitarsObj` is initialized as `new ArrayObject` and an array is given to the constructor. Whenever array operations are performed on `$guitarsObj`, the `ArrayObject` class manages the internal details. In our code we have an object that behaves like an array, and an actual array in the form of `$guitarsArr`. When we pass these two variables to the function `addGuitar()`, a new element is appended within the scope of the function. The function has a single parameter `$arr`, which can be either `array` or `ArrayObject`, declared using a union type declaration.

However, when we look at the results with `var_dump()` only `$guitarsObj` contains a new guitar. The reason for this is the change in scope when execution enters the `addGuitar()` function body. As soon as we make a change to the `$guitarsArr` array PHP creates a copy of it and that copy only exists within the `addGuitar` function. This new array (referenced by the `$arr` function variable) does not exist in global scope and is destroyed once `addGuitar()` completes and control returns to the main program.